Practical exercise 1: Rescue drone task

Óscar Cubeles, Aina Llaneras, María Angélica Jaimes

Abstract

A rescue drone can be crucial in saving people during emergency operations. A drone can quickly localize people in need to be rescued and take them away from harm, while avoiding obstacles and dangerous situations. This report aims to program a simulated rescue drone that can navigate a disaster zone represented by a grid, search for stranded people and take them to a safe zone, all while avoiding obstacles and respecting a safe zone capacity. The main objective is to develop an optimal algorithm that allows the drone to move efficiently through a rescue site through a set of planned actions to rescue as fast as possible all the victims considering a set of restrictions.

1 Analysis of the Problem

The first step to design this planner is to analyze the problem and dissect which components are needed for its implementation.

1.1 Search Space

The search space is going to be a grid of m columns and n rows. We will test the program with an $n \times n$ grid, but it will work with an $m \times n$ grid as well. Each square of the grid is going to be defined by a number.

1.2 Objects

- **Grid**: Each square of the grid has its own number, and therefore is an individual object.
- **Drone**: The drone will be defined as an object, so there can be more than one.
- **People**: Each person in need of rescuing will be defined as an object.
- **S**: An auxiliary object that will have as many objects as the maximum number of people allowed in the safe zone.

1.3 Predicates

We will be using the following predicates to define our planner:

- **Drone Location (d)**: The drone is located at square d of the grid.
- **Person Location (p, d)**: Person p is located at square d.
- **Safe-Zone (d)**: The safe zone is located at square d.
- **Adjacent** (d_1, d_2): Location d_1 is adjacent to location d_2.
- **Rescued (p)**: Person p has been rescued and is in the safe zone.
- **Carrying (p)**: The drone is carrying person p.
- **Obstacle (d)**: There is an obstacle at location d, therefore the drone cannot access location d.
- **Drone-free**: The drone is not carrying anyone.
- **Spot-free (s)**: The safe zone is not full.
- **Spot (s)**: Dummy for persons in the safe zone; the number of spots is dictated by the maximum number of people allowed in the safe zone.

1.4 Operators

The operators (or actions) defined in our planner are the following:

- **Move**: The drone moves from position d_1 to position d_2. The drone can only move to an adjacent position, and it cannot be obstructed by an obstacle.
- **Pick up**: The drone, in square d, picks up person p in the same square d. The drone and the person must be in the same position, and the drone must be free.
- **Drop off**: The drone drops off person p in the safe zone. It can only drop the person at the safe zone, it must be carrying the person, and the safe zone cannot be full. Afterward, the person will be considered rescued and the drone will be free again. This action will also increase the number of people in the safe zone.
- **Empty safe zone**: When the safe zone has reached its maximum capacity, one person is dismissed in order to fit new rescues, and find a solution to the problem.

2 PDDL Implementation

This chapter addresses the implementation of a PDDL (Planning Domain Definition Language) domain designed for the rescue of stranded people using drones. A detailed explanation of the key elements of the domain, the possible actions, and how the planner can use them to generate an efficient rescue plan is presented. Additionally, a problem instance is included and analyzed to illustrate the practical functioning of the domain.

2.1 Domain definition

The PDDL domain `drone-rescue` specifies the fundamental actions and predicates necessary for a drone to carry out the rescue of people in a predefined environment. The key elements of this domain are analyzed below.

```
(define (domain drone-rescue)
  (:requirements :strips)
```

The predicates define the state of the environment, including the location of the drone, the position of the people, and other elements such as obstacles and safe zones.

```
  ;; Predicates
  (:predicates
    (drone-location ?d)           ; The drone is at location ?d
    (person-location ?p ?d)       ; The person ?p is stranded at lo-cation ?d
    (safe-zone ?d)                ; Location ?d is a safe zone
    (adjacent ?d1 ?d2)            ; Locations ?d1 and ?d2 are adja-cent
    (rescued ?p)                  ; The person ?p has been rescued
    (carrying ?p)                 ; The drone is carrying the person
    (obstacle ?d)                 ; Location ?d is an obstacle
    (drone-free)                  ; The drone is not occupied
    (spot-free ?s)                ; There is a free spot in safe-zone
    (spot ?s)                     ; Spot in the safe zone
  )
```

The domain defines the actions the drone can execute, including move-ment between locations, picking up people, and dropping them off in safe zones.

- **Move**: Allows the drone to move from one location to an adjacent one, provided there are no obstacles.

```
;; Action: Move
(:action move
  :parameters (?from ?to)
  :precondition (and
    (drone-location ?from)
    (adjacent ?from ?to)
    (not (obstacle ?to))  ;; Can't move to a location that is an obstacle
  )
  :effect (and
    (not (drone-location ?from))
    (drone-location ?to)
  )
)
```

- **Pickup**: Allows the drone to pick up a person from a location, provided the drone is not already carrying someone.

```
;; Action: Pickup a person
(:action pickup
  :parameters (?p ?d)
  :precondition (and
    (drone-location ?d)
    (person-location ?p ?d)
    (drone-free)         ;; Ensure the drone is not carrying anyone
    (not (obstacle ?d))  ;; Can't pick up from an obstacle location
  )
  :effect (and
    (not (person-location ?p ?d))
    (carrying ?p)        ;; Drone is now carrying the person
    (not (drone-free))   ;; Drone is no longer free
  )
)
```

- **Dropoff**: Allows the drone to drop off a person at a safe zone. The location must be marked as safe and have available space.

```
;; Action: Dropoff a person at a safe zone
(:action dropoff
  :parameters (?p ?d ?s)
  :precondition (and
    (drone-location ?d)
    (safe-zone ?d)   ;; Drop only in a safe zone
    (carrying ?p)    ;; Only drop if carrying a person
    (not (obstacle ?d))  ;; Safe zone cannot be an obstacle
    (not (drone-free))
    (spot ?s)
    (spot-free ?s)
  )
  :effect (and
    (rescued ?p)     ;; Mark person as rescued
    (not (carrying ?p)) ;; The drone no longer carries the person
    (drone-free)     ;; Drone is now free to pick up another person
    (not (spot-free ?s))
  )
)
```

- **Empty Safe Zone**: Resets a safe zone by marking it as available for new rescues once it is full.

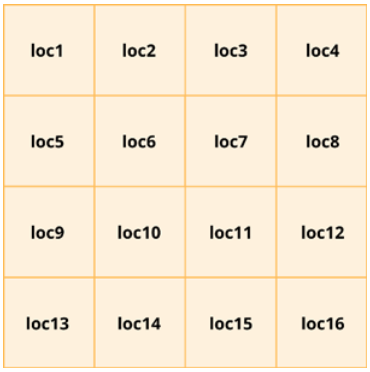


Figure 1: Layout of the rescue environment

```
;; Action: Empty the safe zone when it is full
(:action empty-safe-zone
  :parameters (?s)
  :precondition (and
    (spot ?s)
    (not(spot-free ?s))
  )
  :effect (and
    (spot-free ?s)
  )
)
```

2.2 Problem Definition

The problem instance `rescue-mission-4x4-with-obstacles` describes a specific rescue mission in a 4x4 grid world. This instance defines the initial state of the drone, the people to be rescued, the location of obstacles, and the adjacency relationships between locations.

```
(define (problem rescue-mission-4x4-with-obstacles)
  (:domain drone-rescue)
```

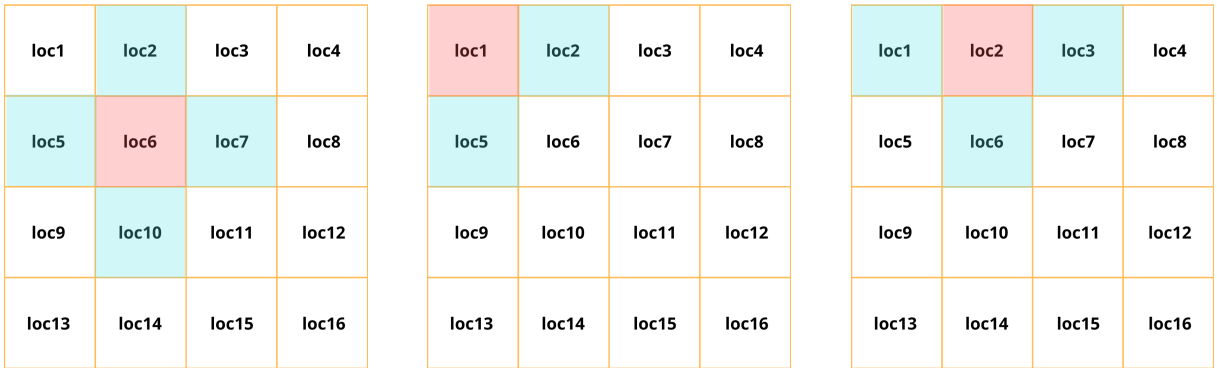
The problem specifies objects such as locations, the drone, the people to be rescued, and the available landing zones. In this example, there are 16 locations arranged in a 4x4 grid, one drone (`drone1`), four people to be rescued, and three available landing spots (`S1`, `S2`, `S3`).

The figure 1 represents the layout of the rescue environment. It consists of 16 distinct locations (labeled `loc1` to `loc16`), arranged in a 4x4 grid. This layout corresponds to the definitions used in the PDDL problem instance. The grid structure helps to visualize the relationships and movement possibilities for the drone as it navigates between locations to rescue stranded individuals.

```
  ;; Objects: drone, people, and locations
  (:objects
    loc1 loc2 loc3 loc4   ;; Row 1 (0-indexed row)
    loc5 loc6 loc7 loc8   ;; Row 2 (1-indexed row)
    loc9 loc10 loc11 loc12 ;; Row 3 (2-indexed row)
    loc13 loc14 loc15 loc16 ;; Row 4 (3-indexed row)
    drone1 - drone          ;; The drone
    person1 person2 person3 person4 - person ;; People to rescue
    S1 S2 S3
  )
```

The initial state defines where the drone and the stranded people are, where the obstacles and safe zones are located, and which locations are adjacent. Obstacles are introduced to make some locations inaccessible, and adjacency relationships describe how the locations are connected for movement.

The figure 2a illustrate the adjacency relationships between different locations in the grid. Locations highlighted in blue (`loc2`, `loc5`, `loc7`, and `loc10`) are adjacent to `loc6`, allowing the drone to move



(a) Locations adjacents to loc6 (b) Locations adjacents to loc1 (c) Locations adjacents to loc2.

Figure 2: Structure of Adjacent locations

between them. The red cell (`loc6`) represents the location being analyzed for adjacency. Likewise, figure 2b and figure 2c represent the locations adjacent to `loc1`, which corresponds to a corner, and `loc2`, which is located along an edge.

```
;; Initial state: Drone and people locations, safe zones, obstacles, and adjacency
  (:init
    ;; Drone starting location (example starting location)
    (drone-location loc1)
    (drone-free)
    ;; People stranded locations
    (person-location person1 loc4)   ;; Person 1 in the last column of row 0
    (person-location person2 loc5)   ;; Person 2 in the first column of row 1
    (person-location person3 loc10)  ;; Person 3 in the second column of row 2
    (person-location person4 loc3)  ;; Person 3 in the second column of row 0

    ;; Safe zones
    (safe-zone loc14)  ;; Safe zone in column 2, row 3
    (spot S1)
    (spot S2)
    (spot S3)

    (spot-free S1)
    (spot-free S2)
    (spot-free S3)

    ;; Obstacles
    (obstacle loc7)    ;; Obstacle in column 3, row 1
    (obstacle loc11)   ;; Obstacle in column 3, row 2
    (obstacle loc16)   ;; Obstacle in column 4, row 3

    ;; Adjacency relationships
    ;; Row 1
    (adjacent loc1 loc2) (adjacent loc2 loc1)
    (adjacent loc2 loc3) (adjacent loc3 loc2)
    (adjacent loc3 loc4) (adjacent loc4 loc3)
    (adjacent loc1 loc5) (adjacent loc5 loc1)
    (adjacent loc2 loc6) (adjacent loc6 loc2)
    (adjacent loc3 loc7) (adjacent loc7 loc3)
    (adjacent loc4 loc8) (adjacent loc8 loc4)

    ;; Row 2
    (adjacent loc5 loc6) (adjacent loc6 loc5)
    (adjacent loc6 loc7) (adjacent loc7 loc6)
```

```
    (adjacent loc7 loc8) (adjacent loc8 loc7)
    (adjacent loc5 loc9) (adjacent loc9 loc5)
    (adjacent loc6 loc10) (adjacent loc10 loc6)
    (adjacent loc7 loc11) (adjacent loc11 loc7)
    (adjacent loc8 loc12) (adjacent loc12 loc8)

    ;; Row 3
    (adjacent loc9 loc10) (adjacent loc10 loc9)
    (adjacent loc10 loc11) (adjacent loc11 loc10)
    (adjacent loc11 loc12) (adjacent loc12 loc11)
    (adjacent loc9 loc13) (adjacent loc13 loc9)
    (adjacent loc10 loc14) (adjacent loc14 loc10)
    (adjacent loc11 loc15) (adjacent loc15 loc11)
    (adjacent loc12 loc16) (adjacent loc16 loc12)

    ;; Row 4
    (adjacent loc13 loc14) (adjacent loc14 loc13)
    (adjacent loc14 loc15) (adjacent loc15 loc14)
    (adjacent loc15 loc16) (adjacent loc16 loc15)
  )
```

The goal of the problem is for the drone to rescue the four people by picking each one up and dropping them off in a safe zone. The goal condition specifies that each person (`person1`, `person2`, `person3`, and `person4`) must be marked as rescued.

```
  ;; Goal: All three persons must be rescued
  (:goal (and
    (rescued person1)
    (rescued person2)
    (rescued person3)
    (rescued person4)
  ))
)
```

3 Testing & Results

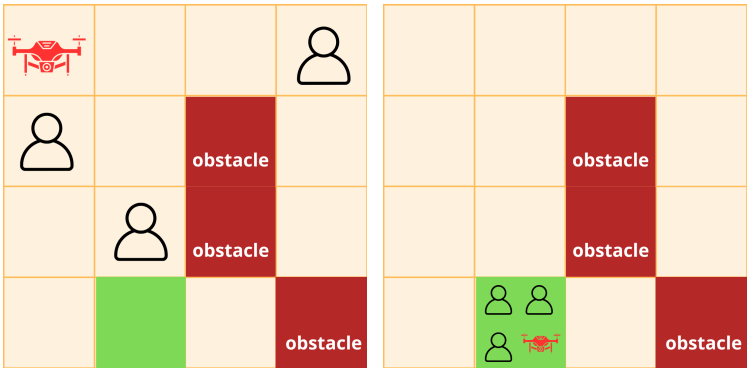
This chapter presents the results obtained through the application of two planners, the BFWS–FF Parser and the DELFI planner, to various planning scenarios. Four test cases have been designed, in which the solutions generated by both planners are compared in terms of necessary steps, routes, and efficiency. Although both methods achieve the same objectives, differences are observed in the sequencing of actions and the number of redundant movements performed in each case. The analysis of these results allows for evaluating the effectiveness of the planners in complex planning scenarios, focusing on resource optimization and the minimization of unnecessary movements.

3.1 Case 1

Figure 3 shows the initial state of Case 1, which consists of a 4x4 grid. The drone is located at position `loc1`, in the top-left corner of the grid. The drone's mission is to rescue three people who are distributed in different positions on the board, while avoiding three obstacles represented by red squares. In the final state, the drone has successfully gathered the three people in the green square corresponding to the safe zone, located in the bottom-left corner, without colliding with the obstacles during its path.

Table 1 lists the actions of the planner in Case 1 for the BFWS–FF Parser Planner and the DELFI Online Planner.

In Case 1, the BFWS–FF Parser and DELFI Planner both solve the problem with different approaches but result in identical step counts (22 steps). The BFWS plan picks up person 2 first, followed by person



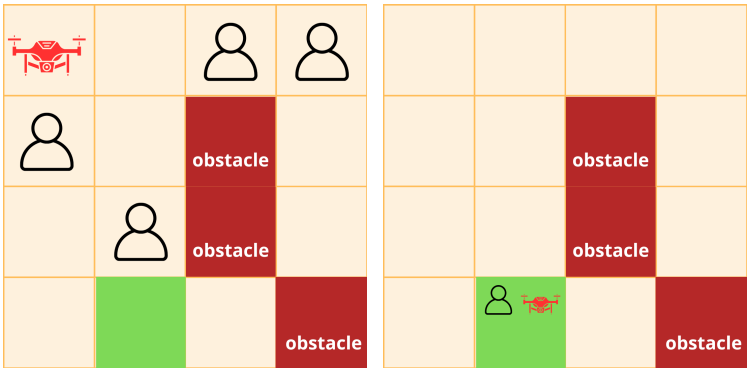
(a) Initial State (b) Goal State

Figure 3: Case 1 Scheme

3 and then person 1, involving more backtracking and movement between locations. In contrast, the DELFI plan starts with person 1, minimizing unnecessary movements and following a more direct path. Although both plans have the same number of steps, we could say that DELFI is slightly more efficient in its route execution.

Table 1: Actions of the planner in Case 1

BFWS –FF Parser Planner	**DELFI Online Planner selection for Cost-Optimal Planner**
1. (move loc1 loc5)	1. move loc1 loc2
2. (pickup person2 loc5)	2. move loc2 loc3
3. (move loc5 loc9)	3. move loc3 loc4
4. (move loc9 loc10)	4. pickup person1 loc4
5. (move loc10 loc14)	5. move loc4 loc3
6. (dropoff person2 loc14 s1)	6. move loc3 loc2
7. (move loc14 loc10)	7. move loc2 loc6
8. (pickup person3 loc10)	8. move loc6 loc10
9. (move loc10 loc14)	9. move loc10 loc14
10. (dropoff person3 loc14 s2)	10. dropoff person1 loc14 s1
11. (move loc14 loc10)	11. move loc14 loc10
12. (move loc10 loc6)	12. move loc10 loc6
13. (move loc6 loc2)	13. move loc6 loc5
14. (move loc2 loc3)	14. pickup person2 loc5
15. (move loc3 loc4)	15. move loc5 loc6
16. (pickup person1 loc4)	16. move loc6 loc10
17. (move loc4 loc3)	17. move loc10 loc14
18. (move loc3 loc2)	18. dropoff person2 loc14 s2
19. (move loc2 loc6)	19. move loc14 loc10
20. (move loc6 loc10)	20. pickup person3 loc10
21. (move loc10 loc14)	21. move loc10 loc14
22. (dropoff person1 loc14 s3)	22. dropoff person3 loc14 s3



(a) Initial State (b) Goal State

Figure 4: Case 2 Scheme

3.2 Case 2

Figure 4 presents the initial state of a more complex scenario. The grid remains 4x4, and the drone is still located at position loc1 (top-left corner). However, unlike the previous scenario, there are four people who need to be rescued, which adds a higher level of difficulty. Additionally, the three obstacles remain, which the drone must avoid while carrying out its rescue mission. In the final state, it is expected that only one person and the drone will remain in the safe zone, as this area had to be empty in a previous step to ensure the maximum capacity of the safe zone.

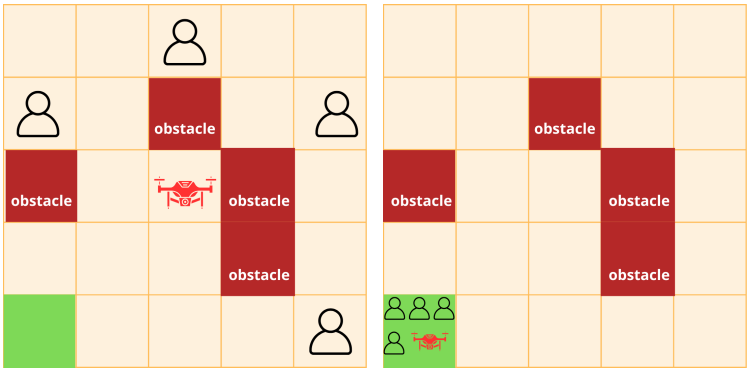
Table 2 lists the actions of the planner in Case 2 for the BFWS–FF Parser Planner and the DELFI Online Planner.

In Case 2, the BFWS–FF Parser and DELFI Planner again arrive at the same solution in terms of tasks, but with different routes and sequencing. Both plans involve moving between locations to pick up and drop off four individuals in 33 steps. The BFWS plan picks up person 2 first, followed by person 3, person 4, and finally person 1, with significant back-and-forth movement, especially between loc10 and loc14. The DELFI plan, however, begins by picking up person 1 and follows a more organized sequence with fewer redundant movements.

Additionally, in contrast to Case 1, in this case, the safe zone is freed before the drop-off of the last person. A relevant insight from the planners is that the BFWS empties the safe zone just before the drop-off, whereas the DELFI planner empties the safe zone just after dropping off the third person.

Table 2: Actions of the planner in Case 2

BFWS –FF Parser Planner	**DELFI Online Planner selection for Cost-Optimal Planner**
1. (move loc1 loc5)	1. move loc1 loc2
2. (pickup person2 loc5)	2. move loc2 loc3
3. (move loc5 loc9)	3. move loc3 loc4
4. (move loc9 loc10)	4. pickup person1 loc4
5. (move loc10 loc14)	5. move loc4 loc3
6. (dropoff person2 loc14 s1)	6. move loc3 loc2
7. (move loc14 loc10)	7. move loc2 loc6
8. (pickup person3 loc10)	8. move loc6 loc10
9. (move loc10 loc14)	9. move loc10 loc14
10. (dropoff person3 loc14 s2)	10. dropoff person1 loc14 s1
11. (move loc14 loc10)	11. move loc14 loc10
12. (move loc10 loc6)	12. move loc10 loc6
13. (move loc6 loc2)	13. move loc6 loc2
14. (move loc2 loc3)	14. move loc2 loc3
15. (pickup person4 loc3)	15. pickup person4 loc3
16. (move loc3 loc2)	16. move loc3 loc2
17. (move loc2 loc6)	17. move loc2 loc6
18. (move loc6 loc10)	18. move loc6 loc10
19. (move loc10 loc14)	19. move loc10 loc14



(a) Initial State (b) Goal State

Figure 5: Case 3 Scheme

BFWS –FF Parser Planner	DELFI Online Planner selection for Cost-Optimal Planner
20. (dropoff person4 loc14 s3)	20. dropoff person4 loc14 s2
21. (move loc14 loc10)	21. move loc14 loc10
22. (move loc10 loc6)	22. move loc10 loc6
23. (move loc6 loc2)	23. move loc6 loc5
24. (move loc2 loc3)	24. pickup person2 loc5
25. (move loc3 loc4)	25. move loc5 loc6
26. (pickup person1 loc4)	26. move loc6 loc10
27. (move loc4 loc3)	27. move loc10 loc14
28. (move loc3 loc2)	28. dropoff person2 loc14 s3
29. (move loc2 loc6)	29. empty-safe-zone s1
30. (move loc6 loc10)	30. move loc14 loc10
31. (move loc10 loc14)	31. pickup person3 loc10
32. (empty-safe-zone s1)	32. move loc10 loc14
33. (dropoff person1 loc14 s1)	33. dropoff person3 loc14 s1

3.3 Case 3

Figure 5 presents the initial state of Case 3, a more complex scenario. The grid is 5x5, with the drone now located at position loc13 (in the center of the grid) and the safe zone positioned in the bottom-left corner. In this case, four people need to be rescued, which corresponds to the maximum number of people allowed in the safe zone. Additionally, the number of obstacles increases to four, which the drone must avoid while carrying out the rescue mission. In the final state, it is expected that the four rescued people and the drone will be in the safe zone.

Table 3 lists the actions of the planner in Case 3 for the BFWS–FF Parser Planner and the DELFI Online Planner.

In Case 3, both the BFWS–FF Parser and DELFI Planner complete the same task, but DELFI does it more efficiently, using 4 fewer steps (50 steps versus BFWS's 54). The BFWS plan picks up person 2, followed by person 4, person 1, and finally person 3, with substantial backtracking, especially between loc21 and loc22. DELFI, on the other hand, optimizes the sequence by picking up person 3 earlier and reducing redundant movements, particularly in the final stages.

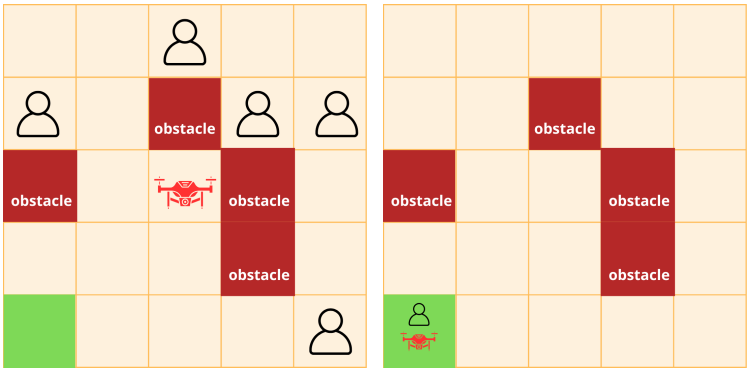
Table 3: Actions of the planner in Case 3

BFWS –FF Parser Planner	DELFI Online Planner selection for Cost-Optimal Planner
1. (move loc13 loc12)	1. move loc13 loc12
2. (move loc12 loc7)	2. move loc12 loc7
3. (move loc7 loc6)	3. move loc7 loc6
4. (pickup person2 loc6)	4. pickup person2 loc6
5. (move loc6 loc7)	5. move loc6 loc7

BFWS –FF Parser Planner	DELFI Online Planner selection for Cost-Optimal Planner
6. (move loc7 loc12)	6. move loc7 loc12
7. (move loc12 loc17)	7. move loc12 loc17
8. (move loc17 loc22)	8. move loc17 loc16
9. (move loc22 loc21)	9. move loc16 loc21
10. (dropoff person2 loc21 s1)	10. dropoff person2 loc21 s3
11. (move loc21 loc22)	11. move loc21 loc22
12. (move loc22 loc23)	12. move loc22 loc23
13. (move loc23 loc24)	13. move loc23 loc24
14. (move loc24 loc25)	14. move loc24 loc25
15. (pickup person4 loc25)	15. pickup person4 loc25
16. (move loc25 loc24)	16. move loc25 loc24
17. (move loc24 loc23)	17. move loc24 loc23
18. (move loc23 loc22)	18. move loc23 loc22
19. (move loc22 loc21)	19. move loc22 loc21
20. (dropoff person4 loc21 s2)	20. dropoff person4 loc21 s2
21. (move loc21 loc22)	21. move loc21 loc22
22. (move loc22 loc23)	22. move loc22 loc23
23. (move loc23 loc24)	23. move loc23 loc24
24. (move loc24 loc25)	24. move loc24 loc25
25. (move loc25 loc20)	25. move loc25 loc20
26. (move loc20 loc15)	26. move loc20 loc15
27. (move loc15 loc10)	27. move loc15 loc10
28. (move loc10 loc5)	28. pickup person3 loc10
29. (move loc5 loc4)	29. move loc10 loc15
30. (move loc4 loc3)	30. move loc15 loc20
31. (pickup person1 loc3)	31. move loc20 loc25
32. (move loc3 loc2)	32. move loc25 loc24
33. (move loc2 loc7)	33. move loc24 loc23
34. (move loc7 loc12)	34. move loc23 loc22
35. (move loc12 loc17)	35. move loc22 loc21
36. (move loc17 loc22)	36. dropoff person3 loc21 s1
37. (move loc22 loc21)	37. move loc21 loc16
38. (dropoff person1 loc21 s4)	38. move loc16 loc17
39. (move loc21 loc22)	39. move loc17 loc12
40. (move loc22 loc23)	40. move loc12 loc7
41. (move loc23 loc24)	41. move loc7 loc2
42. (move loc24 loc25)	42. move loc2 loc3
43. (move loc25 loc20)	43. pickup person1 loc3
44. (move loc20 loc15)	44. move loc3 loc2
45. (move loc15 loc10)	45. move loc2 loc7
46. (pickup person3 loc10)	46. move loc7 loc12
47. (move loc10 loc15)	47. move loc12 loc17
48. (move loc15 loc20)	48. move loc17 loc16
49. (move loc20 loc25)	49. move loc16 loc21
50. (move loc25 loc24)	50. dropoff person1 loc21 s4
51. (move loc24 loc23)	
52. (move loc23 loc22)	
53. (move loc22 loc21)	
54. (dropoff person3 loc21 s3)	

3.4 Case 4

Figure 6 presents the initial state of Case 4. The grid remains 5x5, with the drone still located at position loc13 and the safe zone in the bottom-left corner. However, unlike the previous scenario, in this case there are five people who need to be rescued, which adds a higher level of difficulty. In the final state, it is expected that only one person and the drone will remain in the safe zone, as this area had to be



(a) Initial State (b) Goal State

Figure 6: Case 4 Scheme

emptied in a previous step to ensure the maximum capacity of the safe zone.

Table 4 lists the actions of the planner in Case 4 for the BFWS–FF Parser Planner and the DELFI Online Planner.

Table 4: Actions of the planner in Case 4

BFWS –FF Parser Planner	**DELFI Online Planner selection for Cost-Optimal Planner**
1. (move loc13 loc18)	1. move loc13 loc12
2. (move loc18 loc23)	2. move loc12 loc7
3. (move loc23 loc24)	3. move loc7 loc6
4. (move loc24 loc25)	4. pickup person2 loc6
5. (pickup person4 loc25)	5. move loc6 loc7
6. (move loc25 loc24)	6. move loc7 loc12
7. (move loc24 loc23)	7. move loc12 loc17
8. (move loc23 loc22)	8. move loc17 loc16
9. (move loc22 loc21)	9. move loc16 loc21
10. (dropoff person4 loc21 s3)	10. dropoff person2 loc21 s1
11. (move loc21 loc16)	11. move loc21 loc22
12. (move loc16 loc17)	12. move loc22 loc17
13. (move loc17 loc12)	13. move loc17 loc12
14. (move loc12 loc7)	14. move loc12 loc7
15. (move loc7 loc6)	15. move loc7 loc2
16. (pickup person2 loc6)	16. move loc2 loc3
17. (move loc6 loc7)	17. pickup person1 loc3
18. (move loc7 loc12)	18. move loc3 loc2
19. (move loc12 loc17)	19. move loc2 loc7
20. (move loc17 loc16)	20. move loc7 loc12
21. (move loc16 loc21)	21. move loc12 loc17
22. (dropoff person2 loc21 s2)	22. move loc17 loc22
23. (move loc21 loc16)	23. move loc22 loc21
24. (move loc16 loc17)	24. dropoff person1 loc21 s2
25. (move loc17 loc12)	25. move loc21 loc22
26. (move loc12 loc7)	26. move loc22 loc23
27. (move loc7 loc2)	27. move loc23 loc24
28. (move loc2 loc3)	28. move loc24 loc25
29. (pickup person1 loc3)	29. move loc25 loc20
30. (move loc3 loc2)	30. move loc20 loc15
31. (move loc2 loc7)	31. move loc15 loc10
32. (move loc7 loc12)	32. move loc10 loc9
33. (move loc12 loc17)	33. pickup person5 loc9
34. (move loc17 loc16)	34. move loc9 loc10
35. (move loc16 loc21)	35. move loc10 loc15

BFWS –FF Parser Planner	DELFI Online Planner selection for Cost-Optimal Planner
36. (dropoff person1 loc21 s1)	36. move loc15 loc20
37. (move loc21 loc22)	37. move loc20 loc25
38. (move loc22 loc23)	38. move loc25 loc24
39. (move loc23 loc24)	39. move loc24 loc23
40. (move loc24 loc25)	40. move loc23 loc22
41. (move loc25 loc20)	41. move loc22 loc21
42. (move loc20 loc15)	42. dropoff person5 loc21 s3
43. (move loc15 loc10)	43. move loc21 loc22
44. (pickup person3 loc10)	44. move loc22 loc23
45. (move loc10 loc15)	45. move loc23 loc24
46. (move loc15 loc20)	46. move loc24 loc25
47. (move loc20 loc25)	47. move loc25 loc20
48. (move loc25 loc24)	48. move loc20 loc15
49. (move loc24 loc23)	49. move loc15 loc10
50. (move loc23 loc22)	50. pickup person3 loc10
51. (move loc22 loc21)	51. move loc10 loc15
52. (dropoff person3 loc21 s4)	52. move loc15 loc20
53. (move loc21 loc22)	53. move loc20 loc25
54. (move loc22 loc23)	54. move loc25 loc24
55. (move loc23 loc24)	55. move loc24 loc23
56. (move loc24 loc25)	56. move loc23 loc22
57. (move loc25 loc20)	57. move loc22 loc21
58. (move loc20 loc15)	58. dropoff person3 loc21 s4
59. (move loc15 loc10)	59. empty-safe-zone s1
60. (move loc10 loc9)	60. move loc21 loc22
61. (pickup person5 loc9)	61. move loc22 loc23
62. (move loc9 loc4)	62. move loc23 loc24
63. (move loc4 loc3)	63. move loc24 loc25
64. (move loc3 loc2)	64. pickup person4 loc25
65. (move loc2 loc7)	65. move loc25 loc24
66. (move loc7 loc12)	66. move loc24 loc23
67. (move loc12 loc17)	67. move loc23 loc22
68. (move loc17 loc16)	68. move loc22 loc21
69. (move loc16 loc21)	69. dropoff person4 loc21 s1
70. (empty-safe-zone s1)	
71. (dropoff person5 loc21 s1)	

As in all the previous cases, both planners complete the same task. However, DELFI does so with 2 fewer steps (69 steps compared to BFWS's 71). The BFWS plan starts by picking up person 4 first, followed by person 2, person 1, and so on, resulting in substantial backtracking and repetitive movements, particularly around loc21 and loc25. The DELFI plan is more efficient, starting with person 2 and organizing the remaining pickups to minimize redundant movements. As a result, DELFI achieves the same objectives with fewer steps and a more streamlined sequence of actions.

Another insight is that similar to Case 2, the BFWS planner empties the safe zone just before the last drop-off of the person; however, the DELFI planner does it just after dropping off the fourth person.

4 Conclusions

This exercise has successfully completed its original goal, which was to design an efficient algorithm for a rescue drone to save people from a danger site while avoiding obstacles and respecting the safe zone constraints.

During the tests, two different planners have been used: the BFWS—FF Parser planner and the DELFI planner, to evaluate the optimality of the solutions. Both planners have managed to complete the objectives of the rescue; however, the DELFI planner has proven to be more efficient. In the initial tests on the 4x4 grid, there were not many differences between the planners. Both used the same number of steps to reach the goal, although they did not coincide in all steps taken. For instance, DELFI emptied

the safe zone as soon as it reached its maximum capacity, while BFWS emptied it when a new arrival could not fit in. However, in the latter tests where the 5x5 grid was used, DELFI used fewer steps than BFWS to reach the goal. DELFI has optimized the algorithm while BFWS has just provided a possible solution.

Another important aspect of the project is the handling of the safe zone capacity. To achieve a viable solution, it has been assumed that when the safe zone reaches its maximum capacity, it empties to accommodate new rescues. This may not be realistic, as resources may be limited and handling rescued people may be slower than the drone bringing in new individuals. Regarding the safe zone, the project uses "spots" instead of a number to denote the number of people that can fit inside, in order to introduce capacity constraints.

All in all, the algorithm has been proven to work, and regarding the planners used, DELFI is the faster and more efficient one in terms of steps taken.